# ENGE707 - Assignment

## Kaggle Competition Report

Student Name: Fateh Bhular

Student ID: 23217987

**ENGE707 - Assignment** ..... **1**

Kaggle Competition Report ..... 1

**Task 1: Dataset Exploration & Understanding** ..... **3**

1.1 Classification Problem and Target Classes ..... 3

1.2 Dataset Structure ..... 3

1.3 Class distribution and Dataset Balance ..... 4

1.4 Representative Images and Visual Variation ..... 5

1.5 Image Properties and Data Quality ..... 5

1.6 Intended use of the Dataset ..... 7

**Task 2: Data Preprocessing and Pipeline Preparation** ..... **8**

2.1 Image Conversion and Resizing ..... 8

2.2 Normalisation and Tensor Conversion ..... 9

2.3 Label encoding and Consistency ..... 10

2.4 Reproducible design and Leakage Prevention ..... 11

2.5 Training Data Augmentation ..... 12

2.6 Dataset Objects and Data Loaders ..... 12

**Task 3: Model selection and Pipeline Development** ..... **13**

3.1 Model Selection and Justification ..... 13

3.2 Baseline CNN and ResNet18 Architecture ..... 14

3.3 Loss Function and Hyperparameter Selection ..... 15

3.4 Training and Validation Pipeline Implementation ..... 15

3.5 Prediction Pipeline and Kaggle Readiness ..... 16

**Task 4: Training Strategy and Overfitting/Underfitting** ..... **16**
4.1 Training Setup and Strategy ..... 16
4.2 Learning Curves for CNN and ResNet18 ..... 17
4.3 Overfitting and Underfitting Analysis ..... 18
4.4 Generalisation Strategies and Training Decisions ..... 18
**Task 5: Model Evaluation and Kaggle Submission** ..... **19**
5.1 Validation Evaluation and Model Comparison ..... 19
5.2 Class Performance and Confusion Matrix ..... 20
5.3 Test Predictions and Submission Preparation ..... 21
5.4 Kaggle Submission and Results ..... 21
5.5 Validation and Kaggle Performance Comparison ..... 22
**Task 6: Results Analysis and Improvement Strategies** ..... **22**
6.1 Interpretation of Results ..... 22
6.2 Misclassifications and Difficult Classes ..... 23
6.3 Pipeline effectiveness and Completed experiments ..... 24
6.4 Evaluation Plan and Recommended Improvements ..... 25
6.5 Strengths, Weaknesses and Final Reflection ..... 25
**References** ..... **26**

# Task 1: Dataset Exploration & Understanding

## 1.1 Classification Problem and Target Classes

This project is an image-classification task that requires supervised deep learning. The dataset provided contains testing and training sets of both cats and dogs. During training, the model will look at an image and needs to choose one class from the possible 8 classes for that image. For example, the image can be classified as a CatEye or a DogEye, but not both.



Figure 1.1.1: Cat photo from Kaggle

The dataset contains images of both cat and dog species, where the images can be the face, eye, mouth, or nose of the animal. This makes up 8 possible classes from which the model can choose: CatEye, DogEye, CatFace, DogFace, CatMouth, DogMouth, CatNose, DogNose. The target class is categorical, making this assignment a multi-class classification problem. The model needs to learn the fine-grained visual differences that are needed to recognise cats and dogs apart, and their facial features from each other. The main goal of the model is to achieve a high classification accuracy, which is the official evaluation metric measured by the Kaggle competition.

## 1.2 Dataset Structure

```
Training-label shape: (63660, 2)
Sample-submission shape: (29443, 2)
```

Figure 1.2.1: Image count for train/test sets

The provided dataset contains 63,660 training images and 29,443 testing images, for a total of 93,103 images in both sets. There were four components provided in the dataset: Training images, training labels in CSV format, testing images, and the sample_submission CSV. Both the sample_submission and train_label CSV files have two columns: image_id and class, where train_label maps each image ID to its class for supervised learning. The sample_submission shows what the submission format should be like, displaying the row order and structure.

| Component | Rows | Purpose |
|---|---|---|
| Training images | 63,660 | Model training |
| Testing images | 29,443 | Kaggle predictions |
| train_labels.csv | 63,660 | Training labels |
| sample_submission | 29,443 | Submission format |

## 1.3 Class distribution and Dataset Balance

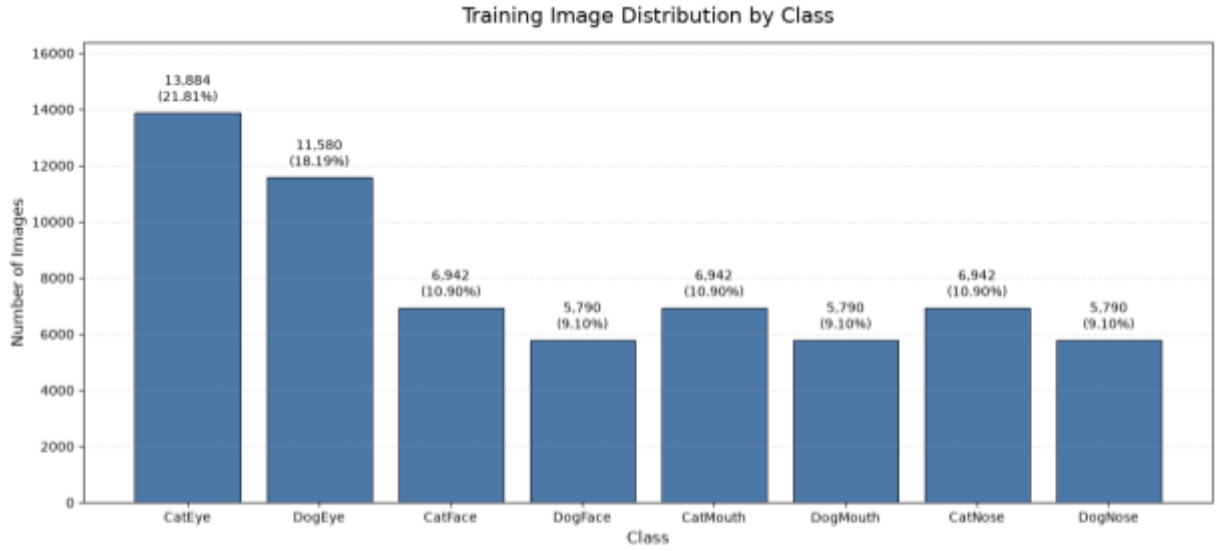



Figure 1.3.1: Distribution of classes

It is important to know the distribution of classes in the dataset, because it gives us a clue about what is being used to train the model. From Figure 1.3.1, CatEye and DogEye have the highest distributions, at 21.81% and 18.19%. Both of these classes have double the number of images as the face, mouth and nose classes. The dataset also contains 20% more cat images than dog images, and the largest cat class has 2.4 times the number of images as the smallest dog class. Therefore, the dataset is moderately imbalanced.

## 1.4 Representative Images and Visual Variation

Some images can be from the same class, but could still look completely different. For example, the lighting levels could be different, with one photo in the sunlight and the other in the shade. The angle of the photo could also be different, with some being low-angled and the others from a high angle. There are several other variables, like fur colour, image sharpness, image sizes and backgrounds, that cause images from the same class to look different.

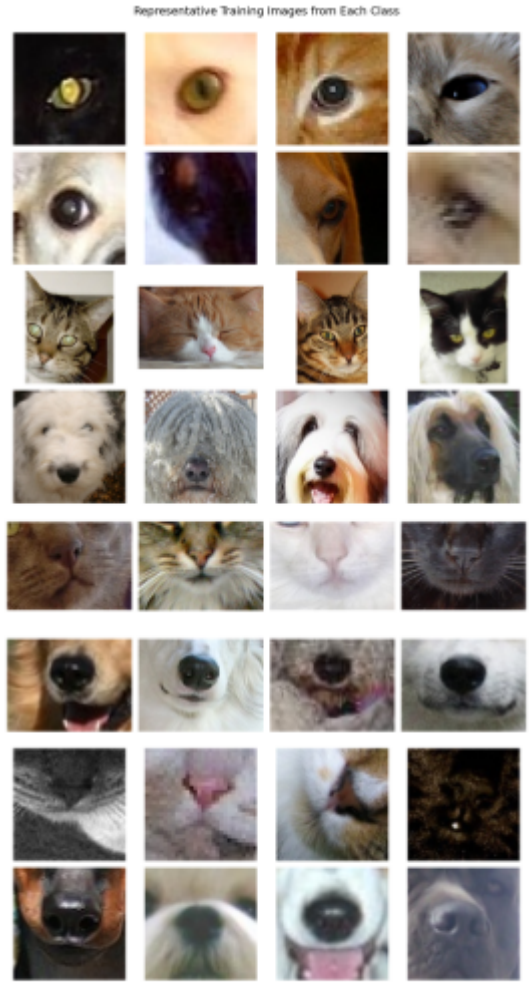



Figure 1.4.1: Representive sample images

Figure 1.4.2 shows how 4 images from every class were picked as representative images. These images were selected using a random seed of 42, making this sample selection reproducible. Images from the same class could look completely different, and images from different classes could look exactly the same. Cats and dogs have similar eyes and colours, so a photo with the same lighting levels and zoom could be confusing for the model to differentiate. Another variable that changes with each image is the height and width in pixels. Some images will have to be padded, which is an approach that avoids removing details that help identify which class the image belongs to. These are all variables that the model has to learn to differentiate throughout training, so that it can successfully identify the animal and facial region for unseen images.

## 1.5 Image Properties and Data Quality

The provided dataset contained a total of 93,103 images from all classes in both the training and testing sets. All of these images could be opened, and there were no corrupted or unreadable files, meaning no files were removed. All of the images were confirmed to use RGB, so there was no format conversion needed.

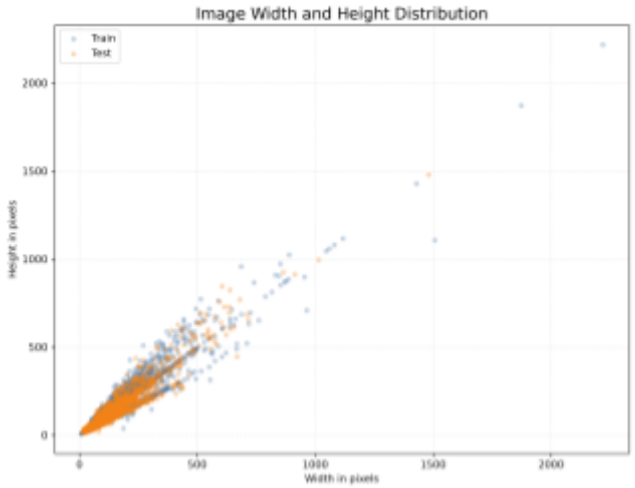



Figure 1.5.1: Image size distribution graph

The sizes of images in the training set were examined, and it was identified that there was a significant number of size combinations. The sizes of images ranged from 4x4 to 2,219x2,219, which means that a fixed input size will have to be selected during data processing. From the evidence in Section 10 in Notebook 1 in the project repository, 40x40, 48x48, and 42x42 were the most common sizes used for images.

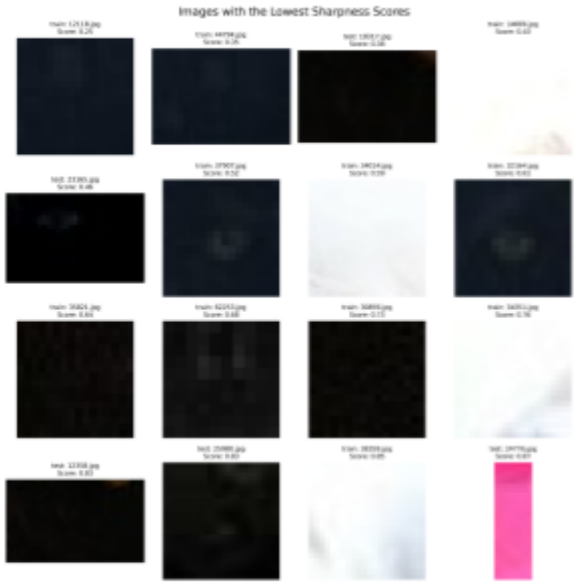



Figure 1.5.2: Low sharpness images

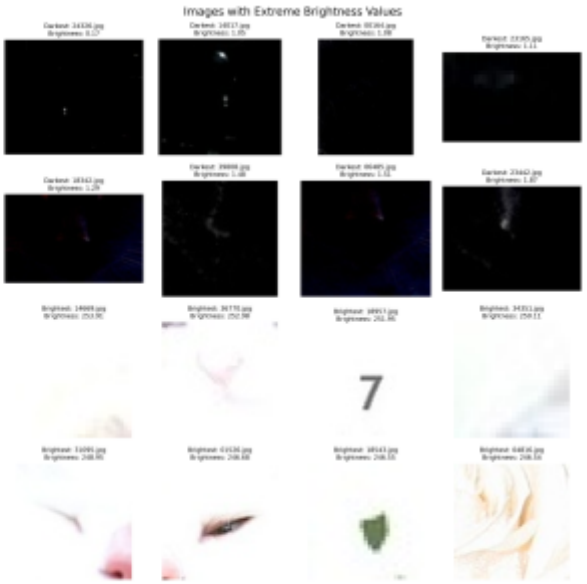



Figure 1.5.3: Extreme brightness images

Figures 1.5.2 and 1.5.3 show the low-sharpness and extremely bright images that were included in the dataset. While these images are harder to interpret for the model, they were kept for training because the sharpness score is only used for screening. The sharpness score shouldn't remove any image for the score alone, so no images under these categories were removed.




Figure 1.5.4: Duplicate images

A SHA-256 hash was used to determine if there were any duplicates in the dataset. There were 15 duplicate groups identified in the training set, and 62 groups in the testing set. There was a total of 77 duplicate groups, but none of them was shared between the two sets. Duplicates could cause data leakage if one copy is in the training subset while the other is in the validation subset. Therefore, the duplicate images should be grouped together so they stay in the same subset, preventing data leakage.

## 1.6 Intended use of the Dataset

CSV manifests will split the original dataset without touching it. There will be a training subset which is fed to the model to learn, and a validation subset which the model uses to try predict the classes without knowing the target. **Stratification** is used to split the training dataset using an **80/20 split**. This ensures that the original distribution of images in the original training set is carried over to the subsets, making the two subsets' environments similar to the original. The training subset is used to update the model's weights, and the validation subset will measure the performance on the images with unseen labels. After training, the model will go though the testing set and try predict the classes of the images. These predictions will be saved and submitted to kaggle for the final classification accuracy score.

# Task 2: Data Preprocessing and Pipeline Preparation

## 2.1 Image Conversion and Resizing

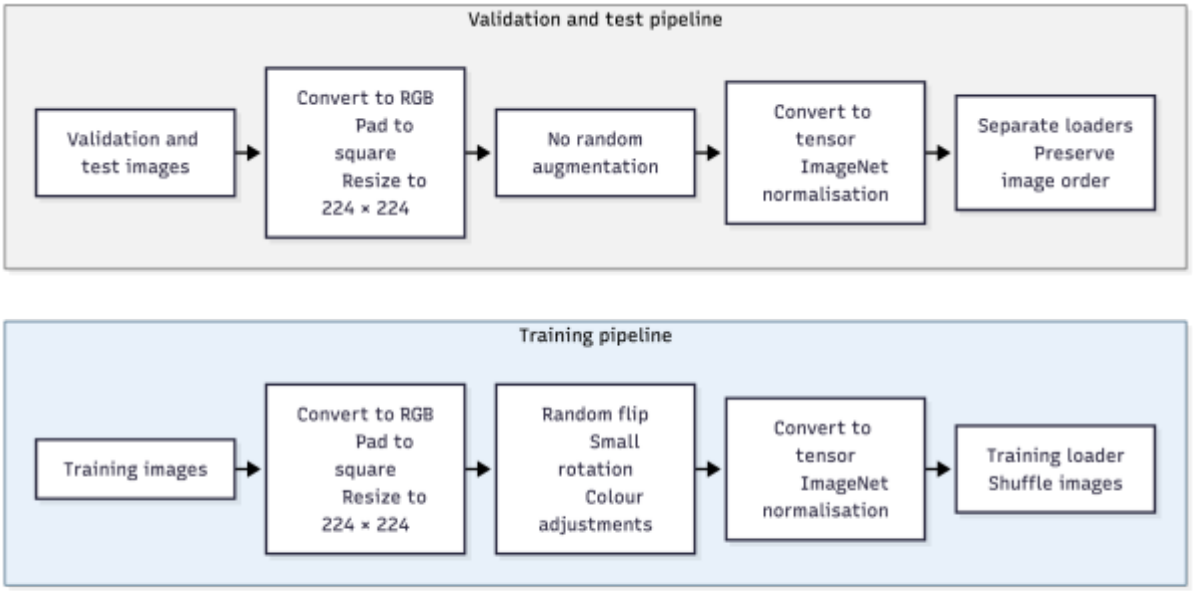



Figure 2.1.1: Training and validation/test pipelines

The images from the original dataset came with different widths and heights as mentioned in section 1.5. This was an issue because different sized images cannot be in the same CNN batch, as it requires all images to have the same dimensions. To pass the images into the CNN, each image was resized to 224x224 pixels, and for a safety check, converted into an RGB format (*Figure 1.5.4*).

There are lots of different ways that images could be resized, and several of these approaches were tested. I ended up using the black-padding approach because it generated the best confidence and accuracy when it came to correct predictions

| | method | accuracy | macro_f1 | errors | mean_confidence | non_square_images | non_square_accuracy | high_padding_images | high_padding_accuracy | accuracy_change_vs_baseline |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | baseline_black_pad | 99.6230% | 0.9966 | 48 | 0.9514 | 4403 | 99.7958% | 3208 | 99.7817% | +0.0000% |
| 1 | horizontal_flip_tta | 99.6230% | 0.9967 | 48 | 0.9524 | 4403 | 99.7958% | 3208 | 99.7817% | +0.0000% |
| 2 | baseline_mean_pad_ensemble | 99.6151% | 0.9966 | 49 | 0.9344 | 4403 | 99.7729% | 3208 | 99.7505% | -0.0079% |
| 3 | baseline_reflect_ensemble | 97.9108% | 0.9786 | 266 | 0.8670 | 4403 | 94.8444% | 3208 | 93.0443% | -1.7122% |
| 4 | imagenet_mean_pad | 97.5259% | 0.9732 | 315 | 0.8895 | 4403 | 93.7315% | 3208 | 91.5159% | -2.0971% |
| 5 | baseline_direct_resize_ensemble | 96.8033% | 0.9634 | 407 | 0.8644 | 4403 | 91.6421% | 3208 | 88.6463% | -2.8197% |
| 6 | baseline_center_crop_ensemble | 92.8841% | 0.9124 | 906 | 0.8546 | 4403 | 80.3316% | 3208 | 73.1129% | -6.7309% |
| 7 | reflect_pad | 79.6576% | 0.6580 | 2590 | 0.9065 | 4403 | 42.0622% | 3208 | 20.5864% | -19.9654% |
| 8 | direct_resize | 79.6576% | 0.6613 | 2590 | 0.9237 | 4403 | 42.0622% | 3208 | 20.5552% | -19.9654% |
| 9 | resize_center_crop | 78.9114% | 0.6578 | 2665 | 0.9317 | 4403 | 40.0863% | 3208 | 18.2470% | -20.7116% |

Figure 2.1.2: Results of several padding approaches




Figure 2.1.3: Examples of original vs padded

After padding was applied to the images, they were resized to 224x224 using a bilinear interpolation approach. This is a method that approximates the value at a certain point within a grid by using the coordinates of values of four other grid points (*GeeksforGeeks, 2024*). This is highly applicable in image processing, and is mainly used for image resizing, image rotation, and zoom.

Figure 2.2.1 shows the comparison of the original images vs after the padding and resizing was applied. Some of them went though both padding and resizing like the second and third pictures, while others had only resizing applied like the first and fourth images.

## 2.2 Normalisation and Tensor Conversion

After padding and resizing, images are converted into a PyTorch tensor. A tensor is the format that PyTorch uses to pass an image to the CNN. It has three values in its shape: number of colour channels, image height, and image width. The pixel values from 0–225 are normalised to a decimal value between 0 and 1 during conversion. This is then normalised using the mean and standard devisiation shown in Figure 2.3.1 (*the values were provided from ImageNet*). These values are different for each colour because they have different ImageNet distributions. For each colour channel, the mean is subtracted from the pixel value, and then tie result of that is divided by the channel's standard deviation.

| **Colour Channel** | **Mean** | **Standard deviation** |
|---|---|---|
| Red | 0.485 | 0.229 |
| Green | 0.456 | 0.224 |
| Blue | 0.406 | 0.225 |

Figure 2.2.1: Table of ImageNet normalisation values for each RGB channel

```python
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

training_transform = transforms.Compose(
    [
        SquarePad(),
        transforms.Resize(
            (IMAGE_SIZE, IMAGE_SIZE),
            interpolation=InterpolationMode.BILINEAR,
        ),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomRotation(
            degrees=10,
            interpolation=InterpolationMode.BILINEAR,
            fill=(0, 0, 0),
        ),
        transforms.ColorJitter(
            brightness=0.15,
            contrast=0.15,
            saturation=0.10,
        ),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=IMAGENET_MEAN,
            std=IMAGENET_STD,
        ),
    ]
)
```

Figure 2.2.2: Code implementation

The code from Figure 2.3.3 shows the preprocessing steps that were applied to each training image. It also includes random adjustments using horizontal flipping and rotation for learnability. After the the images have been converted into a PyTorch tensor, it is normalised using the mean and standard deviation from ImageNet. All of these steps happen in this section of code in notebook 2 in the project files.

## 2.3 Label encoding and Consistency

```python
CLASS_NAMES = [
    "CatEye",
    "DogEye",
    "CatFace",
    "DogFace",
    "CatMouth",
    "DogMouth",
    "CatNose",
    "DogNose",
]

CLASS_TO_INDEX = {
    class_name: index
    for index, class_name in enumerate
    (CLASS_NAMES)
}
```

Figure 2.3.1: Class label mapping

The Pytorch classification model uses numerical target values, so the original training labels that were stored as class names were assigned labels in 0-7. There was no particular order in these labels, they just represented a class so the model could use them. The new encoded values were stored in a new column called “label_index”, while the original names are still kept so they can be read. A reverse mapping was also created because the Kaggle submission requests the class names. Everything is saved in a CSV file so the same class order can be used throughout the assignment.

## 2.4 Reproducible design and Leakage Prevention

The training dataset was split into two subsets: training and validation. This was done using an 80/20 split, where the training subset would update the model's weights, and the validation set would be used to measure the model's performance. StratifiedGroupKFold was used to perform the first split with 5 folds, shuffling enabled, and a random seed of 42. Stratification makes it so that the distribution of the subsets is approximately that of the original training set.

The same resizing approach, ImageNet values, and class mapping is used across the entire assignment, and the same random seed of 42 is used whenever it is needed. This confirms that any processes in this assignment can be reproduced by another person or machine. These processes make this workflow repeatable, but the results could be different slightly depending on the machines themselves.

Data leakage is when the model has access to information that can influence the training process and make the results look better than it actually is. To avoid this, I implemented checks that made sure training and validation images didn't share the same duplicates or image IDs. I split the training and validation as separate CSV files so that the different models could use them. This also provided a check for duplicate values for avoiding data leakage, but it doesn't confirm that related images or resized images are separated.

| Dataset | Images | Purpose | Protection |
|---|---|---|---|
| Training subset | 50,928 | Update model weights | Uses only images assigned to training |
| Validation subset | 12,732 | Compare models and select checkpoints | Excluded from weight updates; no random augmentation |
| Kaggle test set | 29,443 | Generate submission predictions | Excluded from training, model selection, and hyperparameter tuning |

## 2.5 Training Data Augmentation

Data augmentation is the process of creating small changes to training images. I implemented this method so that the model could learn to recognise images using different orientations and lighting. This is applied after the square padding and resizing, but before the tensor conversion and normalisation. Only training images had augmentation applied to them. Validation resulst were used to measure the model's performance, and test images were reserved for created the submission predictions.

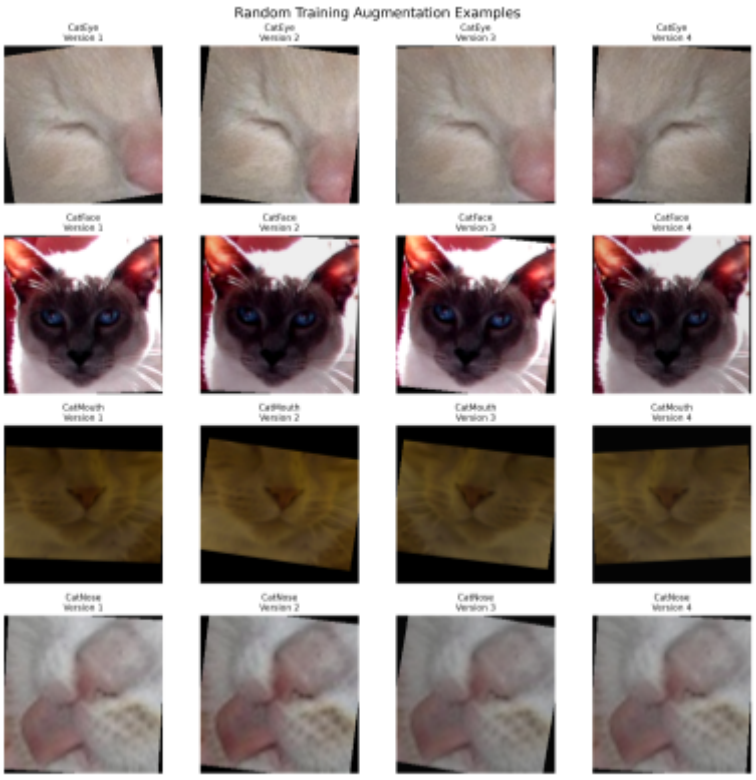



Figure 2.5.1: Examples of Random Augmentation

## 2.6 Dataset Objects and Data Loaders

The custom PyToch dataset used opens the image from the set and applies the image processing. For labelled images, the numeric class label is returned used the mapping from section 2.3. The images are processed in memory, and don't change in the original provided dataset, so a separate copy isn't used. A Pytorch DataLoad is an object that organises images into different batches. The training images are shuffled between every epoch so that their ordering is changed. The validation and test subsets keep their order, so that the predictions can be linked to the image ID's provided. The test predictions follow the order that is in sample_submission.csv.

# Task 3: Model selection and Pipeline Development

## 3.1 Model Selection and Justification

After learning about different deep learning architectures, I decided to use CNNs for this assignment. Convolutional neural networks are used to learn visual patterns from images provided to them. The initial layers learn features like edges and textures, and the layer layers combine the learned features to create complex shapes. This was a good fit for trying to distinguish between dog and cat images, because there are differences in the fur, facial shapes, and backgrounds.

I developed a custom CNN using Pytorch and trained it from scratch. ResNet18 was also used because the weights provided by ImageNet give useful visual features to the model while also keeping the computational cost of training justifiable. Both models use the same training and validation split aswell as the preprocessing.

| layer name | output size | 18-layer | 34-layer | 50-layer | 101-layer | 152-layer |
|---|---|---|---|---|---|---|
| conv1 | 112×112 | 7×7, 64, stride 2 | | | | |
| conv2_x | 56×56 | 3×3 max pool, stride 2 | | | | |
| | | $\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ |
| conv3_x | 28×28 | $\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$ |
| conv4_x | 14×14 | $\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$ |
| conv5_x | 7×7 | $\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ |
| | 1×1 | average pool, 1000-d fc, softmax | | | | |
| FLOPs | | $1.8\times10^9$ | $3.6\times10^9$ | $3.8\times10^9$ | $7.6\times10^9$ | $11.3\times10^9$ |

Figure 3.1.1: ResNet Architectures

## 3.2 Baseline CNN and ResNet18 Architecture




Figure 3.2.1: CNN Architecture

The custom CNN gets 224x224 RGB images and passes it through the convolutional blocks in Figure 3.2.1. The convolutional layers in the architecture learn visual patterns like edges, textures, and shapes. Batch normalisation is used to help stabalise training. ReLU helps the network learn complex relationships, and max pooling reduces the feature-map size.

The different channels keep increasing from 32 all the way to 256 which allows blocks to gradually learn more complicated patterns. The adaptive average pooling layer summarises the final feature maps into one value per feature channel to be used. Dropout helps with preventing overfitting, and the final linear layer gives eight scores, one for each class.

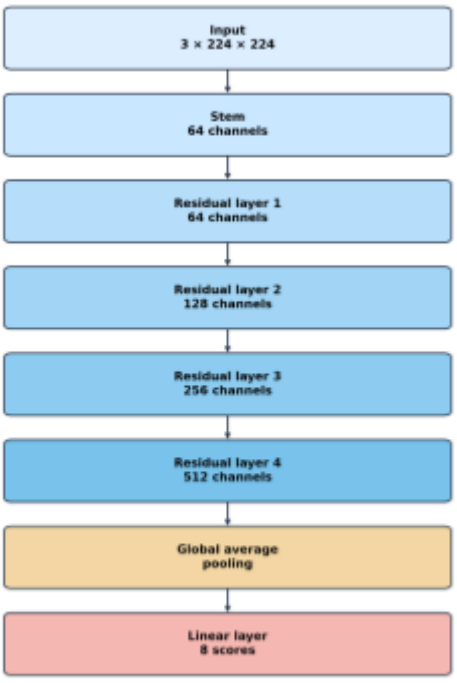



Figure 3.2.2: ResNet18 Architecture

ResNet18 also gets 224x224 RGB images just like CNN. The first layers identify simple patterns like edges and textures, then reduce the width and height of the images internal representations. The four layers of residual blocks learn more complicated patterns that help with recognising features.

ResNet18 starts with pretrained weights from ImageNet, so it already contains patterns that were learned from other images outside this assignment. The pooling summarises the final inputs into 512 values, and the final layer is changed to give a score for the 8 classes. The class with the highest score becomes the prediction.

## 3.3 Loss Function and Hyperparameter Selection

Cross entropy loss was the loss function selected because images have only one correct class. I apple a label smoothing value of 0.05 in ResNet18 so there wouldn't be too much confident fitting. The CNN uses a starting learning rate of 0.001. For Resnet18 only the new classification layer is trained for the first two epochs with the same rate of 0.001. After the entire model is trained for another eight epochs using smaller learning rates. This was applied so that ResNet18 could learn the new classes without making big changes to features it already learned before importing it.

There were difference batch sizes used, as they affected processing speed and memory. The learning-rate scheduling was for adjusting the learning rate during training. These selections were made for starting points because I found them to be reasonable for the start of training. The configuration comparison table is in the project repository.

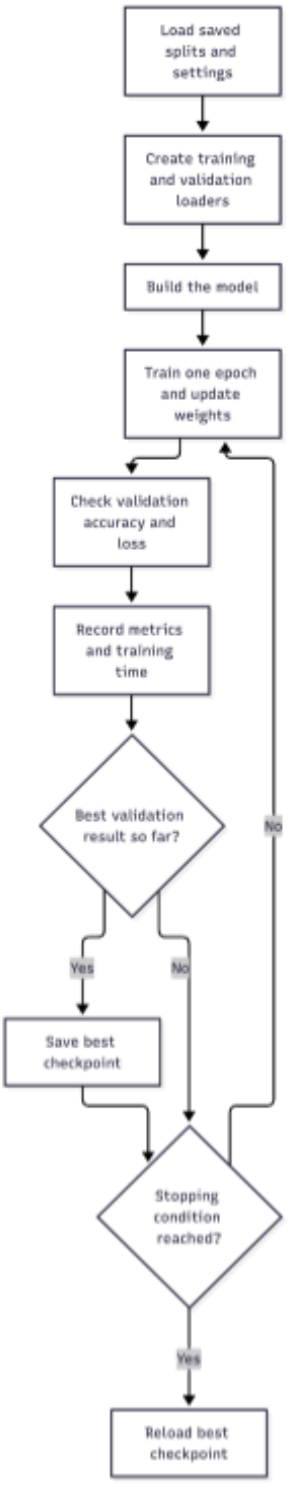



Figure 3.4.1: Training and Validation Pipeline

## 3.4 Training and Validation Pipeline Implementation

The training pipeline used Python and Pytorch. It first loads the training and validation splits, then applies the preprocessing and organises them into batches. The pipeline then shuffles and applies augmentation to training images, while validation images use fixed preprocessing. In training, the model will predict a class, then calculate the error using cross-entropy loss. This is how weights are updated to reduce errors during training. When each epoch is completed, the model goes through the validation subset without updated weights to check its performance. I have made it so the epoch with the highest accuracy will be saved when the model completes training.

### 3.5 Prediction Pipeline and Kaggle Readiness

The prediction pipeline uses the mode's saved checkpoint, this insures that training doesn't need to be repeated. It also checks that the class order matches the label mapping, this is to verify that the model's predictions are not only correct, but use the right mapping. Validation and test images go through the same data preprocessing which includes resizing and normalisaiton.

| | check | result | expected |
|---|---|---|---|
| 0 | Columns are exactly image_id and class | True | True |
| 1 | Row count matches sample submission | True | True |
| 2 | Image ID order is unchanged | True | True |
| 3 | Every image ID is unique | True | True |
| 4 | No prediction is missing | True | True |
| 5 | Every prediction is an official class | True | True |
| 6 | Every test image received one prediction | True | True |
| 7 | Test transform has no random augmentation | True | True |
| 8 | Notebook 5 selected ResNet18 | True | True |

Figure 3.5.1: Prediction checks

There is no updates on the weights when the validation and test sets are used. There are eight class scores for each image, and the class with the highest score will end up being the model's prediction. These predictions follow the order that is provided in sample_submission.csv, and are saved in a CSV with image_id and class columns. I implemented final checks that verify all images have exactly one prediction and that all IDs are unique, this is to ensure the submission will not have any errors.

## Task 4: Training Strategy and Overfitting/Underfitting

### 4.1 Training Setup and Strategy

For all the training and predictions in this assignment, I used kaggle to execute them due to it having better performance than running the processes locally. The CNN was trained from scratch. Training images had random augmentation applied to them, and validation images had fixed processing. I initially planned for 15 epochs for the baseline CNN experiment, but my time guard stopped it at epoch 13. ResNet18 used the pretrained weights that came from ImageNet so it already had learned features. For the first two epochs, only the final classifier layer was trained, after these two, the entire model was trained for 8 more epochs. This allowed ResNet18 to learn how to recognise the 8 categories, before it would start adjusting the previously learned features.


```
{
    "epochs_completed": 13,
    "best_epoch": 13,
    "best_validation_loss": 0.06707265371286453,
    "best_validation_accuracy": 0.9783223374175306,
    "training_accuracy_at_best_epoch": 0.9725691171850456,
    "accuracy_gap_at_best_epoch": -0.005753220232485,
    "verified_validation_loss": 0.06707311217988245,
    "verified_validation_accuracy": 0.9783223374175306,
    "validation_images": 12732,
    "total_training_minutes": 110.00072750001668,
    "stop_reason": "time_guard"
}
```


Figure 4.1.1: Training summary for CNN

| Setting | Custom CNN | ResNet18 |
|---|---|---|
| Starting weights | Random | ImageNet pretrained |
| Planned epochs | Maximum 15 | 2 classifier-only + 8 fine-tuning |
| Completed epochs | 13 | 10 |
| Training time | Approximately 110 minutes | 62.39 minutes |

## 4.2 Learning Curves for CNN and ResNet18

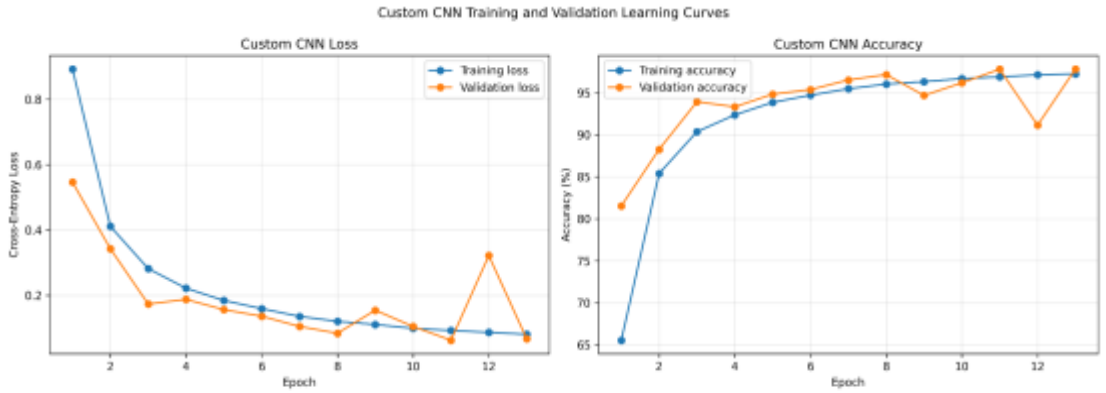



Figure 4.2.1 CNN Baseline Learning Curves

Figure 4.2.1 shows the gradual improvements of the untrained CNN during the baseline experiment. Training accuracy increased from 65.55% to 97.26% during epochs 1 to 13. Validation accuracy increased from 81.53% to 97.83. In epoch 12, the validation accuracy dropped significantly and the validation loss increased significantly. Even though training loss and accuracy remained stable, while validation did not, we cannot confirm that there was overfitting during this epoch.

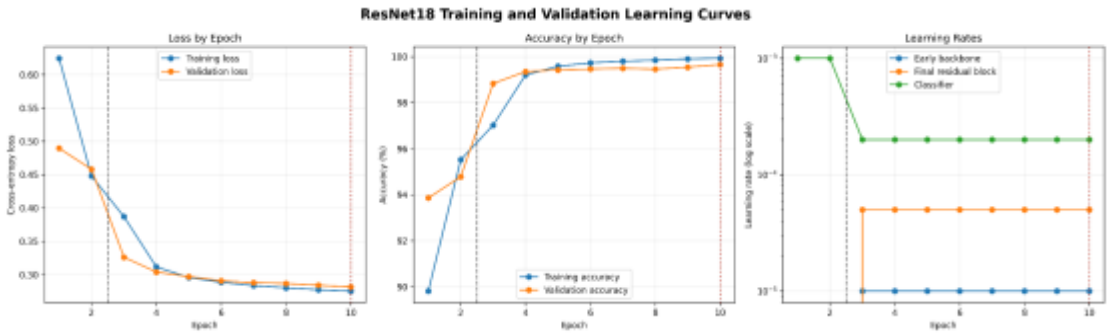



Figure 4.2.2: ResNet18 Learning Curves

Figure 4.2.2 shows the improvements of ResNet18 during classifier training and full network fine-tuning. The validation accuracy improved from 93.86% to 99.65% during epochs 1 to 10. There was a significant improvement from epoch 2 to 3 in both accuracy scores and both loss scores. This supports the evidence that adapting the pretrained features to the dataset was a good step. After epoch 4, the validation performance improved gradually, but the accuracy decreased a little bit at epoch 8. Epoch 10 got the highest validation accuracy which was 99.65%.

## 4.3 Overfitting and Underfitting Analysis

Overfitting is when the model learns the training images too closely, causing it to learn unnecessary data, memorising the data, and performing worse during testing. Underfitting is when the model fails to learn important patterns which causes it to perform worse on both sets of data. None of the models showed underfitting, as they both had good scores for accuracy and loss. During epoch 12, there was a significant drop in validation accuracy (Figure 4.2.1). Validation accuracy also decreased at epoch 9, however it recovered later and improved. These small setbacks do not mean overfitting occured. For ResNet18, the training accuracy was also higher than validation accuracy in the later epochs (Figure 4.2.2). ResNet18 got a 99.93% training accuracy and 99.65% validation accuracy at the end, which is a difference of 0.27 points. But the near-duplicate overlap that was identified overshadows this small gap. The baseline CNN's validation accuracy was higher than its training accuracy at the best checkpoint. This could be due to the augmentation and dropout which could of decreased the training performance.

## 4.4 Generalisation Strategies and Training Decisions

I implemented several approaches to help the model distinguish images and facial regions in the dataset. Random augmentation was applied during preprocessing on the training images, changing the orientations and lighting of those images. Batch normalisation was used to help keep training stable, and weight decay didn't allow excessively large weights. ResNet18 came with pretrained features, which meant it already learned different edges, textures and shapes before training.

| Strategy | Custom CNN | ResNet18 |
|---|---|---|
| Training augmentation | Yes | Yes |

| | | |
|---|---|---|
| Batch normalisation | Yes | Yes |
| Weight decay | 0.0001 | 0.0001 |
| Additional dropout | 0.4 | None |
| ImageNet pretrained weights | No | Yes |
| Label smoothing | No | 0.05 |
| Learning-rate scheduling | Implemented | Implemented |
| Early stopping | Implemented | Implemented |
| Best-checkpoint saving | Yes | Yes |

# Task 5: Model Evaluation and Kaggle Submission

## 5.1 Validation Evaluation and Model Comparison

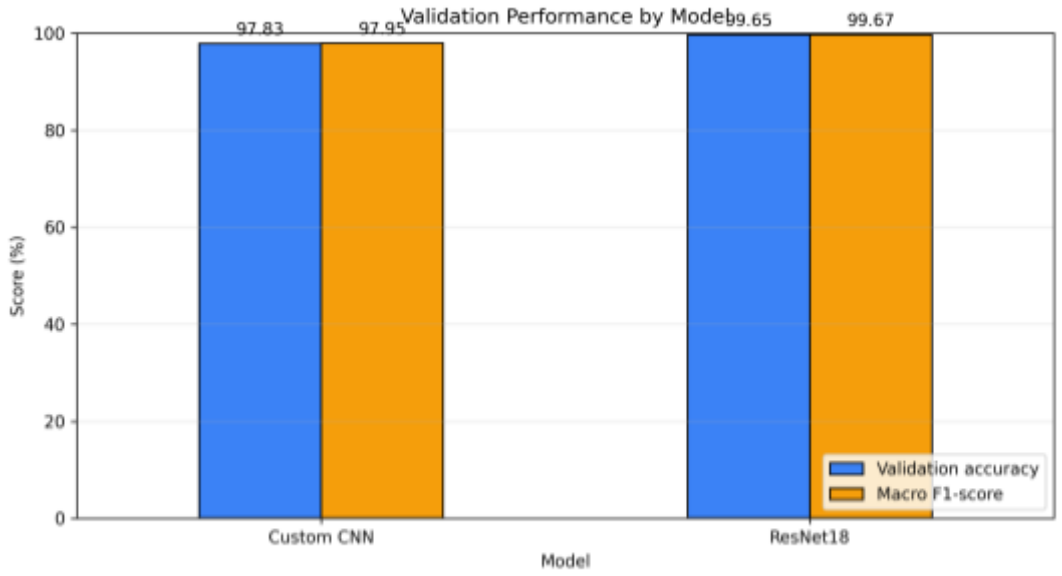



Figure 5.1.1: Validation performance comparison

The validation subset was used to test both models and their performance. During validation, the model's did not update their weights and could only make predictions based off their training. There were three measurements used to compare the model's performance: accuracy, cross-entropy loss,

and macro F1. Accuracy is the percent of correct predictions, while loss calculates how far off the predictions were. Macro F1 measures the performance of the model across all classes. On the validation subset, ResNet18 got 99.65% accuracy and CNN got 97.83% accuracy. ResNet18 also had a lower loss score and higher macro F1 score which means it performed better than CNN overall.

## 5.2 Class Performance and Confusion Matrix

| Class | Precision | Recall | F1-score |
|---|---|---|---|
| CatEye | 0.9953 | 0.9978 | 0.9966 |
| DogEye | 0.9957 | 0.9948 | 0.9952 |
| CatFace | 1.0000 | 0.9986 | 0.9993 |
| DogFace | 1.0000 | 0.9991 | 0.9996 |
| CatMouth | 0.9957 | 1.0000 | 0.9978 |
| DogMouth | 1.0000 | 0.9965 | 0.9983 |
| CatNose | 0.9942 | 0.9899 | 0.9921 |
| DogNose | 0.9940 | 0.9957 | 0.9948 |

Figure 5.2.1: Class-specific scores

The ResNet18 validation results were evaluated based on precision, recall, and F1-score. Precision means how frequently a predicted class was correct, recal shows how many images belonging to a class are correctly identified, and F1 is a combination of both. As shown in figure 5.2.1. The lowest scores across the three columns are 0.9940, 0.9899, and 0.9921. This confirms that the model performed excellently during validation with only several misclassifications.

Custom CNN misclassified images: 276

| | true_class | predicted_class | count |
|---|---|---|---|
| 27 | DogNose | DogEye | 50 |
| 1 | CatEye | DogEye | 49 |
| 11 | CatNose | DogEye | 32 |
| 0 | CatEye | CatNose | 16 |
| 9 | CatNose | CatEye | 15 |
| 14 | DogEye | CatEye | 14 |
| 18 | DogEye | DogNose | 13 |
| 26 | DogNose | CatNose | 9 |
| 28 | DogNose | DogFace | 8 |
| 2 | CatEye | DogNose | 8 |

ResNet18 misclassified images: 44

| | true_class | predicted_class | count |
|---|---|---|---|
| 8 | DogEye | CatEye | 7 |
| 6 | CatNose | DogEye | 5 |
| 4 | CatNose | CatEye | 5 |
| 0 | CatEye | CatNose | 3 |
| 1 | CatEye | DogEye | 3 |
| 5 | CatNose | CatMouth | 3 |
| 10 | DogEye | DogNose | 3 |
| 9 | DogEye | CatNose | 2 |
| 12 | DogMouth | CatMouth | 2 |
| 16 | DogNose | DogEye | 2 |

Figure 5.2.2: Misclassification count comparison

Figure 5.2.2 shows the misclassified images that each model made. ResNet18 made 44 errors across all 12,732 validation images. The most mistakes that ResNet18 misclassified was predicting a DogEye image as CatEye, happening seven times.

## 5.3 Test Predictions and Submission Preparation

ResNet18 was used to make predictions on the testing dataset, after the evaluation proved it to be the better model. Before testing starting, the model's label mapping was checked to make sure there would be no incorrect predictions due to wrong labels. When testing started, the testing images went through the same preprocessing as the validation subset. The steps included adding padding, resizing to 224x224, converting to tessors, and applying normalisation.

For each image, the class with the highest score ended up being the prediction and was recorded. All predictions were saved in the same order that is in the sample_submission.csv file. The final file had two columns just like the sample file, and there were checks implemented. These checks confirmed there were no issues in the submission like duplicate predictions or incorrect labels.

```
check,result,expected
Columns are exactly image_id and class,True,True
Row count matches sample submission,True,True
Image ID order is unchanged,True,True
Every image ID is unique,True,True
No prediction is missing,True,True
Every prediction is an official class,True,True
Every test image received one prediction,True,True
Test transform has no random augmentation,True,True
Notebook 5 selected ResNet18,True,True
```

Figure 5.3.1: Submission checks


```
{
    "selected_model": "ResNet18",
    "checkpoint_epoch": 10,
    "validation_accuracy": 0.9965441407477222,
    "test_images": 29443,
    "prediction_seconds": 1505.403326565,
    "mean_confidence": 0.9005966871233895,
    "minimum_confidence": 0.18007472157478333,
    "predicted_class_counts": {
        "CatEye": 6253,
        "DogEye": 5138,
        "CatFace": 3139,
        "DogFace": 2786,
        "CatMouth": 3396,
        "DogMouth": 3267,
        "CatNose": 2994,
        "DogNose": 2470
    },
    "submission_checks_passed": true,
    "test_data_used_for_training": false,
    "test_data_used_for_model_selection": false
}
```


Figure 5.3.2: Submission details

## 5.4 Kaggle Submission and Results

I submitted the ResNet18 prediction file to the Kaggle submission page, where the final public leaderboard score was 0.9463 or 94.63% classification accuracy. This score is my model's performance on the actual testing set from Kaggle. Figure 5.4.1 shows the summary of the submission file including the prediction class counts, validation accuracy, confidence, and more.




Figure 5.4.1: Leaderboard statistics

### 5.5 Validation and Kaggle Performance Comparison

| Evaluation | Accuracy |
|---|---|
| Original validation | 99.65% |
| Kaggle public leaderboard | 94.63% |

The ResNet18 model that was used had achieved a 99.65% accuracy on the validation subset. However in the real test, it achieved only 94.63%. This is a 5.02% drop which could be due to a number of reasons. The testing set could contain images that are harder to analyse due to sizes, different angles, lighting conditions, etc.

## Task 6: Results Analysis and Improvement Strategies

### 6.1 Interpretation of Results

As shown from the results in Task 5, ResNet18 got a 99.65% accuracy, but during the submission it only received 94.63%. This confirms that the performance was not as expected and that it didn't perform aswell on the unseen data. I performed a split audit which identified 398 pairs that were nearly the same crossing over between the training and validation split. This confirms that images that were similar were shared between the subsets, but it doesn't explain how much the validation accuracy was affected. After retraining the model with perceptual grouping, the validation accuracy remained around the same at 99.62%. This suggests that the similar-images overlap doesn't explain the performance on Kaggle.

Differences in lighting and image features are other aspects that could have caused that 5.02% decrease in accuracy. From this evaluation, I learned that a high validation accuracy does not guarantee that the model will perform the same on unseen data.

## 6.2 Misclassifications and Difficult Classes

From Figure 5.2.2 in section 5, ResNet18 misclassified 44 images. The most frequent misclassification was predicting DogEye images as CatEye, which happened 7 times. The class with the lowest recall rate was CatNose with 98.99%. These misclassifications indicate that facial features from different classes were similar, and the model let those similarities confuse it during validation. Cropping was a preprocessing step that I did not implement, because I predicted that they would cut out important details that help the model distinguish between classes. Some examples of the misclassifications are the following Figure 6.2.1.

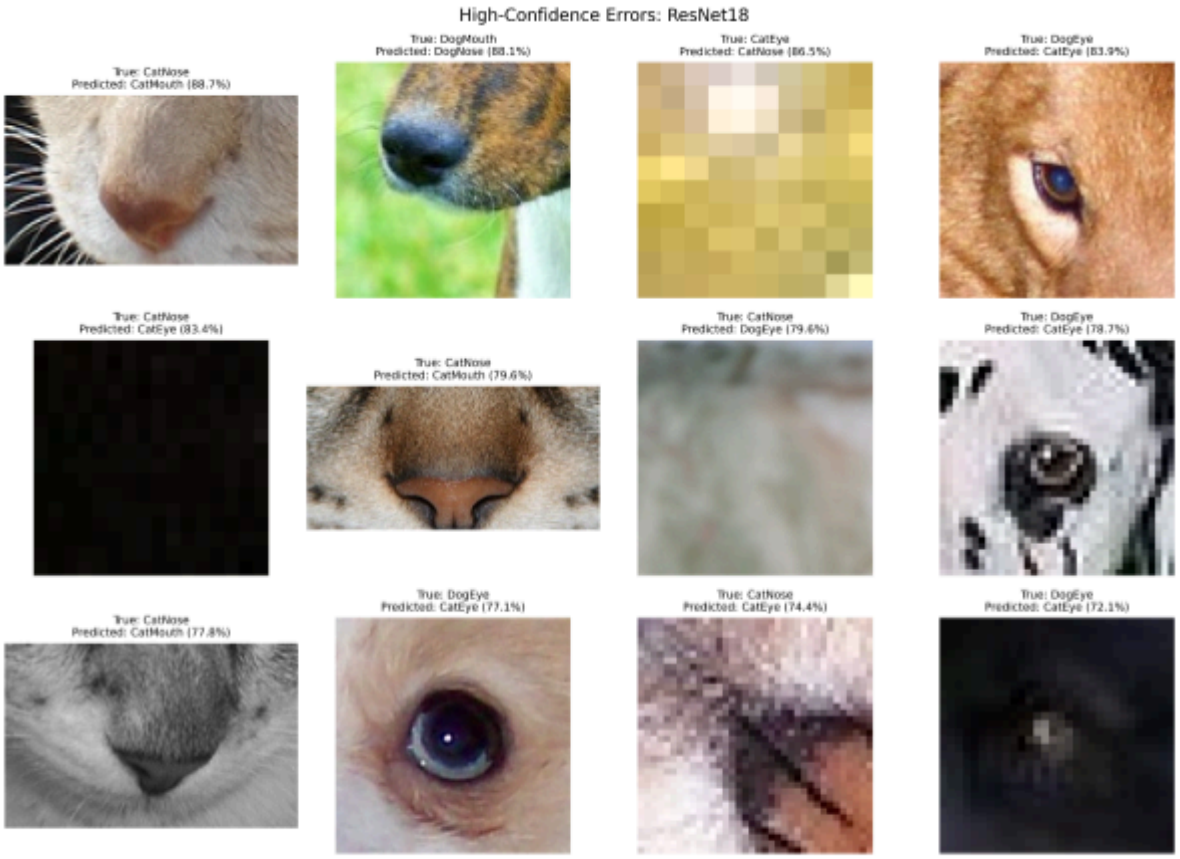



Figure 6.2.1: Representative images of misclassifications

The dark CatNose image that was predicted as Cateye contains barely any details, making the feature harder for the model to identify. Another CatNose example is very pixelated which makes the shape less clear. These are examples of images that were very difficult to predict for the model.

## 6.3 Pipeline effectiveness and Completed experiments

| Experiment | Validation accuracy | Conclusion |
|---|---|---|
| CNN baseline | 97.83% | Good baseline |
| ResNet18 | 99.65% | Performed better than CNN baseline |
| Resnet18 & horizontal-flip tta | 99.68% | Small improvement with less errors |
| Resnet - perceptually grouped split | 99.62% | Reduced near-duplicate overlap |

The CNN baseline got a 97.83% validation accuracy, and the pretrained ResNet18 got 99.65% on the same split. Several pipeline processes were involved in making the validation accuracy scores high: padding, consistent resizing, and normalisation. Padding was used to keep important image details rather than cropping the image. All images were resized to a consistent 224x224 size using bilinear interpolation. Finally normalisation allowed the inputs to be suitable for the model to use. The individual benefits of these processes were not measure separately, so their contribution to the accuracy cannot be identified by themselves.

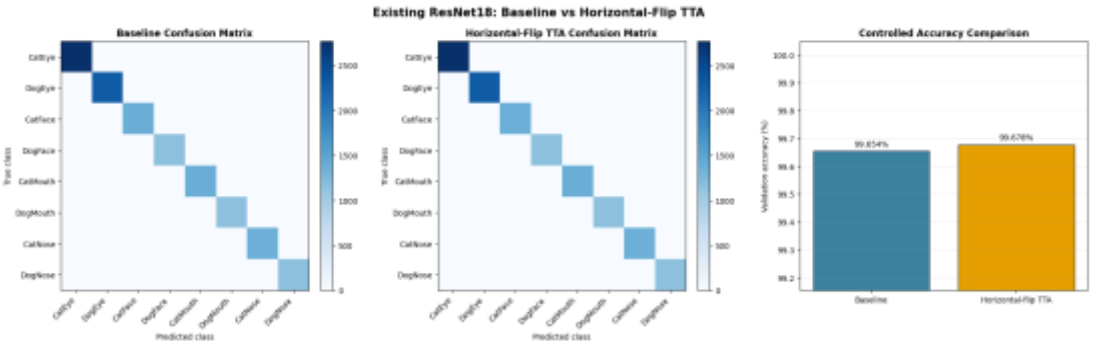



Figure 6.3.1: Confusion matrix comparison of ResNet18 experiments

As an improvement, I implemented horizontal-flip test-time augmentation which ended up increasing the validation accuracy to 99.68%. This change improved 7 of the errors, but also introduced 4 new ones. This means 3 less errors were produced, and that the small improvement didn't confirm the kaggle submission would improve. Another experiment I performed was perceptually grouping, which is when images that look very similar are grouped together. This

reduced the duplicate overlap between the training and validation sets, however the validation images changed so we cannot directly compare this.

## 6.4 Evaluation Plan and Recommended Improvements

The first step I would do to work on this project is to improve the grouping of similar images during the training-validation split. This will reduce the number of duplicate or near-duplicate images crossing between sets. If near duplicate images are shared between sets, the model's validation accuracy could be high because the model has already seen an identical image.

Another experiment I would prioritise is data augmendation testing, where I would test how different augmentations affect validation accuracy. This could provide evidence as to what could be used to improve the model's image recognition.

Lastly I would learn more about the hyperparamaters, and how they could effect validation results at a deeper level. This would be applied by testing small gradual changes, and then comparing the results to see what changes improved the model, and which made it worse.

## 6.5 Strengths, Weaknesses and Final Reflection

The strengths of this image-classification workflow are the preservation of the original data, making the workflow reproducible and auditing every step take, and keeping the best checkpoints during training. ResNet18 performed better than the CNN baseline which indicates that transfer learning was a positive change made to this image-classification project. However, the gap in validation accuracy and the Kaggle submission shows that the original accuracy that I got didn't have the same performance on the unseen test images. The checks performed after the CNN baseline improved certain errors and misclassifications, but some relationships were unidentified even until the end of this assignment.

The main goal of this assignment was to produce an image-classification workflow that could identify the visual differences between 8 different image classes, and correctly predict them. I learned that accuracy does not represent everything about the model, and that every improvement and performance reduction should be analysed.

# References


Siddique, AB. (2026a). *Convolutional neural networks* [Lecture slides]. ENGE707, Auckland University of Technology.

Siddique, AB. (2026b). *Introduction to deep learning* [Lecture slides]. ENGE707, Auckland University of Technology. [Course LMS URL]

Siddique, AB. (2026c). *Learning in deep networks* [Lecture slides]. ENGE707, Auckland University of Technology. [Course LMS URL]

Contributors, P. (2023, January 1). PyTorch documentation. https://docs.pytorch.org/docs/2.14/index.html

GeeksforGeeks. (2026a, May 12). Introduction to Convolution Neural Network. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/

GeeksforGeeks. (2024, September 22). What is Bilinear Interpolation? GeeksforGeeks. https://www.geeksforgeeks.org/maths/what-is-bilinear-interpolation/

12. Common pitfalls and recommended practices. (n.d.). Scikit-learn. https://scikit-learn.org/stable/common_pitfalls.html